Сессионное задание
Основы криптографии
Номер студенческого:
M265897

1. Опишите трехсторонний протокол распределения ключей между парами участников с использованием третьей стороны

Допустим, в сети много пользователей, и любые двое из них могут захотеть переписываться защищённо. Для этого им нужен общий секретный ключ. Если раздать заранее отдельный ключ каждой паре, ключей станет слишком много. Поэтому используют третью сторону а именно, доверенный центр. У каждого пользователя есть только один секретный ключ, общий с центром, а центр по запросу выдаёт общий ключ той паре, которой он нужен.
В протоколе три стороны:
Алиса - это пользователь, который хочет начать защищённую переписку;
Боб - это пользователь, с которым Алиса хочет переписываться;
Центр - это доверенная сторона, он знает личные ключи всех пользователей (свой ключ Алисы и свой ключ Боба) и умеет создавать новые сеансовые ключи. Обоим пользователям приходится ему доверять.
Сам протокол состоит из таких шагов.
Шаг 1. Алиса → Центр. Алиса сообщает центру, что хочет связаться с Бобом. К запросу она добавляет случайное число, которое придумала только что. Оно нужно, чтобы потом убедиться, что ответ пришёл на этот запрос, а не является старым перехваченным сообщением. Этот запрос можно отправить открыто, секретов в нём нет.
Шаг 2. Центр → Алиса. Центр создаёт новый сеансовый ключ для Алисы и Боба. Он отправляет Алисе сообщение которое зашифровано её личным ключом. В нём лежат, сеансовый ключ, имя Боба, случайное число Алисы и ещё одна копия сеансового ключа вместе с именем Алисы, зашифрованная личным ключом Боба. Алиса расшифровывает сообщение и проверяет, что её число на месте и ключ выдан для связи именно с Бобом. Копию для Боба она прочитать не может, так как не знает его ключа.
Шаг 3. Алиса → Боб. Алиса пересылает Бобу его копию сеансового ключа, а Боб расшифровывает её своим личным ключом и узнаёт сеансовый ключ и то, что связь начинает Алиса. Так как эта часть зашифрована ключом, который знают только Боб и центр, Боб понимает, что её подготовил центр.
Шаг 4. Боб → Алиса. Боб придумывает своё случайное число и отправляет его Алисе, зашифровав уже сеансовым ключом. Так он проверяет, что у Алисы действительно есть этот ключ.
Шаг 5. Алиса → Боб. Алиса расшифровывает число, немного изменяет его, к примеру добавляя единицу, снова шифрует

сеансовым ключом и отправляет Бобу. Изменение нужно, чтобы злоумышленник не мог просто переслать Бобу его же сообщение с шага 4. Если Боб получил правильный ответ это означает, что ключ есть у обоих.
Таким образом, у Алисы и Боба появляется общий сеансовый ключ, и дальше они шифруют им свою переписку. Этот ключ знают только Алиса, Боб и центр. Злоумышленник в сети видит только, кто с кем хочет связаться, и зашифрованные сообщения, а сам ключ в открытом виде нигде не передаётся. Для любой другой пары пользователей всё происходит точно так же.
Описанная схема называется протоколом Нидхема–Шрёдера. У неё есть слабое место. Боб не может проверить, что присланная ему копия ключа свежая. Если старый сеансовый ключ когда-то утёк, злоумышленник может снова отправить Бобу старую копию и выдать себя за Алису. Поэтому позже в сообщения стали добавлять метку времени. На этой идее работает протокол Kerberos, который используется, например, для входа в корпоративную сеть Windows.

2. Приведите формальные модели шифров

Формальная модель нужна, чтобы описать шифр математически, не привязываясь к конкретному алгоритму. Тогда про любой шифр можно строго говорить, правильно ли он расшифровывает и насколько он стойкий. Рассмотрим три модели.
1. Алгебраическая модель
Шифр записывают как набор из пяти частей: $\Sigma = (X, K, Y, E, D)$, где:
X - это все открытые тексты, которые можно зашифровать
K - все возможные ключи
Y - все возможные шифртексты
E - правила зашифрования, для каждого ключа k своё правило E_k, которое переводит текст из X в шифртекст из Y
D - правила расшифрования, D_k переводит шифртекст обратно в текст.
Основное требование звучит так: если зашифровать текст и потом расшифровать на том же ключе, должен получиться исходный текст, то есть $D_k(E_k(x)) = x$. Из этого следует, что на одном ключе два разных текста не могут дать один и тот же шифртекст, иначе получатель не поймёт, что ему отправили. Если тексты и шифртексты берутся из одного множества (например, блоки по 128 бит), то каждое E_k просто переставляет элементы этого множества. Так устроены блочные шифры.

Эта модель нужна, чтобы описать, как работает шифр, и проверить, что он правильно расшифровывает. Про стойкость она ничего не говорит.

2. Вероятностная модель

К алгебраической модели добавляют вероятности. Первая. С какой вероятностью встречается каждый открытый текст P(X), например, в русском тексте буква «о» встречается намного чаще, чем «ф». Вторая. С какой вероятностью выбирается каждый ключ P(K), обычно все ключи равновероятны. Ключ выбирают независимо от текста. Зная эти две вещи, можно посчитать, что противник узнаёт об открытом тексте, когда видит шифртекст.

Эта модель нужна для оценки стойкости. В ней Шеннон ввёл понятие совершенного шифра. Это шифр, у которого перехваченный шифртекст вообще ничего не говорит злоумышленнику, $P(x \mid y) = P(x)$. Пример такого шифра - это шифр Вернама, где ключ случайный, длиной с сообщение и используется один раз.

3. Модель секретной системы связи Шеннона

Эта модель описывает не сам алгоритм, а всю систему передачи. В ней есть источник сообщений, источник ключей, от которого ключ по защищённому каналу попадает к отправителю и получателю, шифратор, открытый канал, дешифратор и противник, который перехватывает шифртекст в канале.

Нужна она, чтобы понять, откуда берутся угрозы. По ней видно, что вся секретность держится только на ключе, а сам алгоритм может быть известен противнику. И ещё видно, что для симметричного шифра нужен отдельный защищённый канал, чтобы передать ключ. Эту проблему как раз решают протоколы вроде описанного в вопросе 1.

3. Приведите принципы построения поточных систем шифрования

В поточном шифре текст шифруется по одному символу (биту или байту). Каждый символ складывается по модулю 2 с символом гаммы: $c_i = x_i \oplus \gamma_i$. При расшифровании ту же гамму прибавляют ещё раз и получают исходный текст. Гамму выдаёт генератор который работает от ключа. Поэтому стойкость поточного шифра почти полностью зависит от того, какая получается гамма. Основные принципы построения такие.

Гамма должна быть похожа на случайную, то есть все значения встречаются примерно одинаково часто и следующий символ нельзя предсказать по предыдущим. Это нужно для стойкости, потому что если в гамме есть закономерности, они переходят в шифртекст и по ним можно что-то узнать о тексте. Если гамма по-настоящему

случайная и не повторяется, шифр становится абсолютно стойким, так реализован шифр Вернама.
Период гаммы должен быть большим. Любой генератор когда-нибудь начинает повторяться, и надо, чтобы это происходило намного позже, чем заканчивается текст. Если гамма повторилась, злоумышленник может сложить два куска шифртекста и получить сумму двух открытых текстов, а такую сумму уже можно разобрать, это тоже относится к стойкости.
Генератор должен быть сложным, нелинейным. Простой линейный регистр сдвига сам по себе не подходит, по небольшому куску его гаммы весь регистр легко восстановить. Поэтому несколько регистров объединяют через нелинейную функцию или сдвигают их неравномерно. Ещё важно, чтобы выход генератора не был связан с каким-то одним регистром, иначе их можно взломать по одному. Это нужно, чтобы по известной части гаммы нельзя было вычислить остальную.
Ключ должен быть длинным, и одна и та же гамма не должна использоваться повторно. Сейчас ключ берут от 128 бит, чтобы его нельзя было перебрать. Чтобы на одном ключе не получалась одна и та же гамма, к каждому сообщению добавляют свой вектор инициализации. Например, протокол WEP в Wi-Fi взломали как раз из-за повтора гаммы.
Нужно выбрать способ синхронизации. В синхронном шифре гамма не зависит от шифртекста, так как, ошибка в одном бите портит один бит текста, но если символ потеряется, дальше расшифровывается мусор. В самосинхронизирующемся шифре гамма зависит от предыдущего шифртекста, после сбоя он сам восстанавливается, но ошибка портит несколько символов. Этот принцип нужен, чтобы шифр нормально работал в конкретном канале связи с помехами или с потерями.
Шифр должен быть быстрым и простым в реализации. Это нужно для эффективности, ведь скорость - это главный плюс поточных шифров. Поэтому их используют в связи и на слабых устройствах, например ChaCha20 в TLS и WireGuard.

4. Приведите пример режима шифрования, в котором биективность всех шифрующих отображений базового режима не является необходимой для однозначного расшифрования криптограмм. Ответ обоснуйте

Базовый режим - это режим простой замены (ECB), где каждый блок текста шифруется напрямую отображением E_k. Как раз про эти отображения E_k (для каждого ключа своё) и спрашивается, обязаны ли они быть биективными.

Пример такого режима - режим гаммирования (OFB). Так же работают режим гаммирования с обратной связью (CFB) и режим счётчика (CTR).
В режиме гаммирования блочный шифр не шифрует сам текст, а используется для получения гаммы, так берётся начальное значение S_0 (синхропосылка).
Каждый блок гаммы считается так: $\gamma_i = E_k(S_{(i-1)})$, и этот же блок становится следующим входом: $S_i = \gamma_i$.
Текст складывается с гаммой: $c_i = x_i \oplus \gamma_i$.
Получатель знает ключ и синхропосылку, поэтому сам считает точно такую же гамму той же функцией зашифрования E_k. Потом он получает текст: $x_i = c_i \oplus \gamma_i$.
Почему здесь не нужна биективность:
обратная функция D_k вообще не используется, и отправитель, и получатель только вычисляют E_k;
текст однозначно восстанавливается за счёт операции XOR: если к числу два раза прибавить одну и ту же гамму по модулю 2, получится исходное число;
от E_k нужно только одно, чтобы при одном и том же ключе и входе она всегда выдавала одинаковый результат. Тогда гамма у обоих совпадёт. Даже если E_k для двух разных входов иногда выдаёт одинаковый выход, на расшифрование это не повлияет.
Для сравнения, в режимах ECB и CBC текст подаётся прямо на вход шифра, $c_i = E_k(x_i)$. Чтобы получить x_i обратно, нужно обратить E_k, а если бы два разных блока давали одинаковый шифртекст, то понять, какой из них был отправлен было бы невозможно. Поэтому там биективность обязательна, а в режиме гаммирования нет.

5. Сколько 64-битовых блоков открытого текста искажается при расшифровании криптограммы симметричного блочного шифра, если в шифрованном тексте исказился один бит и используется режим гаммирования с обратной связью по шифртексту, в котором блоки открытого и шифрованного текста имеют длину 8 бит? Ответ обоснуйте

Блочный шифр работает с блоками по 64 бита, а режим гаммирования с обратной связью по шифртексту (CFB) шифрует текст кусками по 8 бит, то есть по одному байту. В шифртексте испортился один бит.
Данный режим работает следующим образом. У отправителя и получателя есть одинаковый регистр на 64 бита, в начале в нём синхропосылка. На каждом шаге, регистр шифруется блочным шифром, и из результата берутся первые 8 бит - это гамма.

Байт текста складывается с гаммой по модулю 2, получается байт шифртекст, регистр сдвигается на 8 бит, и в конец записывается байт шифртекста. Получатель кладёт в регистр тот шифртекст, который ему пришёл. Поэтому если байт пришёл с ошибкой, ошибка попадает и в регистр.
Регистр длиной 64 бита, сдвиг каждый раз на 8 бит. Значит, каждый байт шифртекста лежит в регистре 64 / 8 = 8 шагов, а потом выходит из него.
Теперь посмотрим, что будет, если испорчен один бит в каком-то байте шифртекста.
Сам этот байт расшифруется почти правильно, поскольку гамма для него ещё верная, поэтому в открытом тексте испортится только тот же один бит. Следующие 8 байтов расшифруются неправильно целиком. Пока испорченный байт лежит в регистре, блочный шифр выдаёт другую гамму, а при изменении хотя бы одного бита на входе шифра меняется примерно половина бит на выходе.
После этого испорченный байт выходит из регистра, и дальше всё расшифровывается нормально.
Итого портится 1 + 8 = 9 байтов подряд, то есть 72 бита.
Остаётся перевести это в 64-битовые блоки. В одном таком блоке 8 байтов, поэтому 9 байтов подряд в один блок не влезут, а в два влезут всегда. Значит, ошибка задевает 2 соседних блока по 64 бита. Так искажаются 9 восьмибитовых блоков открытого текста, это 2 блока по 64 бита. Дальше режим сам восстанавливается.

6. Однонаправленная хэш-функция принимает значения из множества двоичных n-мерных векторов. При каком n случайная пара сообщений составляет коллизию с вероятностью, не большей 10^{-9}? Обязательно должен быть приведен ход решения

Хэш - это двоичный вектор длины n, поэтому всего возможных значений хэша $N = 2^n$.
Будем считать хэш-функцию идеальной, для случайного сообщения любое из N значений получается с одинаковой вероятностью, и для разных сообщений значения не зависят друг от друга. Тогда можно использовать модель парадокса дней рождения. Вероятность того, что среди k сообщений хотя бы у двух совпадут хэши,
$P = 1 - (1 - 1/N)(1 - 2/N)\ldots(1 - (k - 1)/N)$.
В задаче всего одна пара сообщений, k = 2, поэтому
$P = 1 - (1 - 1/N) = 1/N = 1/2^n$.

То есть хэш второго сообщения должен случайно совпасть с хэшем первого, а вариантов всего N. По условию эта вероятность не больше 10^{-9}

$1/2^n \le 10^{-9}$

отсюда

$2^n \ge 10^9$.

Логарифмируем по основанию 2:

$n \ge \log_2(10^9) = 9 \cdot \log_2 10$.

$\log_2 10 = \ln 10 / \ln 2 \approx 2{,}3026 / 0{,}6931 \approx 3{,}3219$, значит

$n \ge 9 \cdot 3{,}3219 \approx 29{,}9$.

n должно быть целым, поэтому $n \ge 30$.

Проверяем:

$n = 30$: $1/2^{30} = 1/1\,073\,741\,824 \approx 9{,}3 \cdot 10^{-10}$, меньше 10^{-9}, значит подходит.

$n = 29$: $1/2^{29} = 1/536\,870\,912 \approx 1{,}9 \cdot 10^{-9}$ - больше 10^{-9}, не подходит.

Ответ: $n \ge 30$, то есть длина хэша должна быть не меньше 30 бит.

Это касается только одной случайной пары. Если специально перебирать много сообщений, совпадение найдётся гораздо быстрее, поэтому в реальных хэш-функциях длина намного больше и как правило - это 256 бит.